DIPLOMA EN CIENCIA DE DATOS
E INTELIGENCIA ARTIFICIAL APLICADA

Ingeniería de Datos y MLOps

Semana 2

Diseño y verificación de pipelines de datos

Implementación en Python con apoyo de agentes

Libro teórico y de aplicación

Caso acumulativo: Comercial Horizonte

Prof. Mag. Ing. Mat. Diego Fernández

El propósito de estas dos clases

Convertir el diagnóstico de calidad en un proceso repetible: generar, comprender, ejecutar y revisar un pipeline con reglas explícitas y pruebas.

Material para las Clases 3 y 4. Caso y UM ficticios.

Cómo utilizar este libro

Este volumen desarrolla `Clase_03_Lunes_Python_Generado_por_Agentes.tex` y `Clase_04_Miercoles_Pipeline_Python.tex`, manteniendo el formato del libro de la Semana 1.

La presentación de la Clase 3 y la guía práctica de la Clase 4 conducen dos encuentros de 90 minutos. Este libro ofrece las explicaciones, ejemplos y actividades para preparar la conversación, estudiar después y justificar decisiones. No se propone leerlo íntegramente en voz alta ni ampliar el programa obligatorio con infraestructura adicional.

Caso y alcance. Comercial Horizonte es ficticia. Su CSV tiene 16 registros, seis columnas y tres categorías: `Oficina`, `Tecnologia` y `Hogar`. Las fechas de venta y de ejecución son simuladas; no fijan el calendario del curso. UM significa unidad monetaria ficticia. La semana desarrolla Ingeniería de Datos y fundamentos operativos útiles para MLOps: pruebas, trazabilidad y repetición. No constituye una práctica del ciclo completo de aprendizaje automático: no entrenamos, desplegamos ni monitorizamos un modelo predictivo. El agente implementa y ayuda a revisar el pipeline; obtener código generado no basta como evidencia de aprendizaje.

Forma de trabajo. El agente genera y explica procedimientos; el estudiante define el objetivo, revisa el plan, verifica resultados y conserva evidencia. No se evalúa la escritura manual de Python ni la memorización de comandos. La ejecución puede ser individual o docente observada, pero esa modalidad debe declararse. Una propuesta de código no equivale a una ejecución.

Fuentes del contenido. Se revisaron las dos clases de esta semana, las reglas y el contrato del caso, las políticas v1/v2, la matriz de ocho pruebas, la ficha e instrucciones del analista y la plantilla acumulativa de entrega. Las referencias oficiales del final sustentan el contexto técnico general. Los ejemplos hipotéticos se identifican; los resultados de referencia están separados de las actividades para que no sustituyan la comprobación propia.

Ruta de lectura

Clase teórica: capítulos 1 a 7 y ejemplos seleccionados de 8 y 9: repetición, pruebas y detención.
Clase práctica: capítulos 8 a 11, recuperando el contrato y los ejemplos anteriores.
Después de clase: actividades y apéndices; resolver antes de consultar referencias. Trabajo asincrónico previsto: 2 horas, sin agregar requisitos presenciales.
Producto de la semana: E2: script revisado, tres productos, ocho pruebas y bitácora acumulativa.

Índice general

Cómo utilizar este libro i

1. Del diagnóstico al proceso 1
- 1.1. La pregunta que organiza el trabajo . . . 1
- 1.2. Un análisis puntual y un proceso que se conserva . . . 1
- 1.3. Resultados observables de la semana . . . 2

2. Python y lectura del código 3
- 2.1. Script, instrucciones y ejecución . . . 3
- 2.2. Leer una decisión: condición, motivo y detención . . . 3
- 2.3. Leer una suma y una escritura . . . 4
- 2.4. Una biblioteca estándar suficiente . . . 4

3. Contrato e interfaz 5
- 3.1. Entradas, parámetros y salidas . . . 5
- 3.2. La interfaz no cambia la regla . . . 5
- 3.3. Condiciones de detención . . . 5

4. Leer y validar 6
- 4.1. Leer conserva estructura y procedencia . . . 6
- 4.2. Validar conserva decisiones y motivos . . . 6
- 4.3. El tamaño del maestro no limita el pipeline . . . 7
- 4.4. La conciliación de conteos . . . 7

5. Cálculos y porcentajes 8
- 5.1. La unidad del cálculo . . . 8
- 5.2. Dividir después de sumar . . . 8
- 5.3. Pérdidas y conjunto vacío . . . 8

6. Decimal y presentación 9
- 6.1. Representar un importe antes de operar . . . 9
- 6.2. Calcular, comparar y mostrar . . . 9
- 6.3. Conservar Decimal en un JSON . . . 9

7. Política y clasificación 10
- 7.1. Dos fechas con responsabilidades distintas . . . 10
- 7.2. Elegir una única política válida . . . 10
- 7.3. Un estado informativo . . . 10

8. Guardar productos y evidencia 11
- 8.1. Tres productos con funciones diferentes . . . 11
- 8.2. Una ejecución tiene identidad . . . 11
- 8.3. R1: repetir la misma versión sin regenerarla . . . 11
- 8.4. Abrir los archivos también es revisar . . . 12

9. Pruebas y regresión 13

9.1. Entrada, esperado, observado y evidencia . . . 13
9.2. Agrupar pruebas para comprenderlas en 90 minutos . . . 13
9.3. D1: una detención visible, paso a paso . . . 13
9.4. Lo que la matriz no cubre por sí sola . . . 14
9.5. Corrección mínima y regresión . . . 14

10. Trabajo con agentes **15**
10.1. Una instrucción que hace revisable el trabajo . . . 15
10.2. Revisar con preguntas que buscan fallos . . . 15
10.3. Corregir a partir de una discrepancia . . . 15

11. Práctica, entrega y continuidad **16**
11.1. Una sesión de 90 minutos con productos concretos . . . 16
11.2. Preparación docente y contingencia . . . 16
11.3. Dos horas autónomas, con alcance definido . . . 16
11.4. Qué conserva E2 . . . 17
11.5. Cómo se conecta con la Semana 3 . . . 17

A. Referencia del caso y contraste independiente **18**
A.1. Reglas que permanecen durante las cinco semanas . . . 18
A.2. CSV original del caso . . . 18
A.3. Resultados de referencia: consultar después de calcular . . . 19
A.4. Políticas y estados de referencia . . . 19
A.5. Fixtures pequeños que aíslan una regla . . . 19
A.6. Archivos de prueba listos para preparar . . . 19

B. Actividades y respuestas razonadas **21**
B.1. Actividades para resolver antes de consultar . . . 21
B.2. Respuestas de referencia . . . 21
B.3. Taller de revisión: un fallo que parece razonable . . . 22
B.4. Casos adicionales de lectura crítica . . . 23

C. Plantillas y glosario de trabajo **24**
C.1. Registro de prueba . . . 24
C.2. Bitácora de una ejecución . . . 24
C.3. Vocabulario esencial . . . 24
C.4. Criterios comunes de evaluación . . . 25

Referencias y recursos **26**

Capítulo 1

Del diagnóstico de calidad a un proceso repetible

1.1 La pregunta que organiza el trabajo

En la Semana 1 se decidió qué representa cada campo y qué hacer ante una incidencia. La Semana 2 convierte esas decisiones en una secuencia ejecutable. La pregunta sigue siendo la misma: ¿qué ventas cumplen las reglas y qué margen generan? El cambio es que el procedimiento debe poder repetirse, producir archivos nuevos y explicar cada exclusión.

Un diagnóstico escrito puede decir que V005 aparece dos veces. Un programa debe aplicar esa regla a cualquier repetición exacta que encuentre. Esa generalización exige una condición explícita: comparar los seis campos originales, conservar la primera aparición y registrar el origen de la posterior. Escribir simplemente «eliminar duplicados» deja abiertas decisiones que el contrato ya resolvió.

La repetición del procedimiento permite detectar discrepancias entre lo que se pidió y lo que el código hace. Sin embargo, repetir una operación equivocada no la vuelve correcta. Un programa que promedia porcentajes puede funcionar sin errores de ejecución y conservar un error de negocio en cada repetición. Por eso necesitamos un contrato y pruebas, además de una herramienta.

1.2 Un análisis puntual y un proceso que se conserva

Una respuesta puede resolver la pregunta de hoy. Un pipeline permite volver a recorrer los pasos sin reconstruirlos en cada conversación. Para lograrlo necesita entradas identificadas, reglas explícitas, parámetros, productos esperados y una respuesta prevista ante los fallos.

Pregunta de diseño	Decisión para Comercial Horizonte
¿Qué puede variar?	El CSV compatible, la fecha simulada y la carpeta de salida.
¿Qué se conserva?	La versión revisada del código y el contrato aplicable.
¿Qué debe explicarse?	La aceptación, exclusión o detención y el origen de cada decisión.
¿Cómo se acepta?	Comparando resultados reales con pruebas independientes.

El pipeline de esta semana es determinista: con iguales datos, reglas y parámetros produce los mismos resultados de negocio. El agente que ayuda a escribirlo puede proponer códigos distintos en conversaciones diferentes. Por eso repetir el pipeline significa ejecutar el mismo archivo conservado; no volver a pedir otra implementación.

Qué aporta a MLOps y qué queda fuera

Las pruebas, las versiones, la trazabilidad y el entorno reproducible son bases para operar sistemas de aprendizaje automático. Un ciclo completo añade preparación y evaluación de modelos, despliegue, seguimiento y actualización. La Semana 2 prepara esas capacidades operativas con un proceso de datos acotado. En las semanas siguientes distinguiremos la prueba del cálculo determinista de la evaluación de respuestas del agente analista.

1.3 Resultados observables de la semana

El estudiante debe explicar los cuatro bloques del pipeline; reconocer dónde se detiene un esquema incompatible; justificar las exclusiones; comprobar importes y porcentaje global; distinguir cálculo, política y presentación; y conservar el script, tres productos y ocho pruebas. También deberá volver a ejecutar esa versión y explicar qué debe permanecer igual. La entrega E2 amplía E1. El agente puede escribir el programa, pero la aceptación requiere criterio humano.

Una automatización que conserva un error

El agente propone convertir -2 en 2 y completar un precio faltante con el promedio. El código ejecuta esas instrucciones sin fallar. El producto incumple el contrato: ha creado datos que la fuente no contenía. La corrección consiste en cambiar la implementación para excluir y explicar, no en adaptar las reglas al programa.

Para discutir: ¿Qué diferencia hay entre repetir un cálculo y poder verificarlo?

Idea central

El pipeline transforma reglas acordadas en acciones repetibles. Las pruebas permiten comprobar si esa transformación fue fiel.

Capítulo 2

Comprender Python sin memorizar sintaxis

2.1 Script, instrucciones y ejecución

Un script es un archivo de instrucciones para un intérprete. En el caso, `pipeline.py` recibe entradas, verifica reglas, calcula y produce resultados. El nombre del archivo no demuestra su comportamiento: hay que leer su explicación, localizar los controles y observar una ejecución.

Conviene reconocer tres formas de instrucciones. Una asignación conserva un valor, por ejemplo las ventas de una fila. Una condición decide qué camino seguir, por ejemplo si el precio existe. Una repetición aplica un mismo procedimiento a varios registros. Una función reúne un trabajo con un nombre comprensible, como validar una fila. El objetivo es relacionar estas formas con decisiones del contrato, sin estudiar una lista de símbolos.

El siguiente fragmento es ilustrativo, no un pipeline completo. Su función es vincular variables y fórmulas:

```
from decimal import Decimal
cantidad = 10
precio = Decimal("25")
costo = Decimal("15")
ventas = cantidad * precio
costos = cantidad * costo
margen = ventas - costos
```

El estudiante debería poder decir qué representa cada valor y comprobar 250, 150 y 100 UM. Todavía faltan lectura, validación, agrupación, política y guardado; ejecutar estas líneas no demuestra que se cumplió el contrato completo.

2.2 Leer una decisión: condición, motivo y detención

No se pide escribir Python de memoria. Sí se pide señalar dónde se aplica una regla y anticipar qué ocurrirá. Estos fragmentos suponen variables ya definidas; no son un programa completo ni sustituyen el resto de las validaciones.

```
if encabezado != columnas_esperadas:
    raise ValueError("Esquema incompatible")

motivos = []
if fila["precio_unitario"] == "":
    motivos.append("precio vacio")
# Aqui se agregan tambien los demas motivos encontrados.
if motivos:
    excluidas.append({"origen": origen, "motivos": motivos})
else:
    validas.append(fila)
```

`if` significa «si se cumple esta condición». La sangría identifica las instrucciones que dependen de ella. `!=` compara si dos valores son distintos; `==` compara igualdad. `raise` señala un fallo: el control superior debe detener el flujo y devolver error. No debe capturarlo para seguir calculando como si nada hubiera ocurrido.

`append` agrega un elemento a una lista. Primero se reúnen todos los motivos de una fila; después se registra una sola exclusión. El esquema incompatible afecta al archivo completo. El precio vacío afecta a una fila y tiene una respuesta prevista por el contrato. El fragmento omite duplicados, conflictos de ID, conversión y otras reglas; el estudiante debe buscarlas en el script real.

Para discutir: Antes de ejecutar, completa: «si falta precio, esta fila se ...; si cambia el encabezado, el proceso ...».

2.3 Leer una suma y una escritura

Supongamos que las filas ya fueron validadas y sus importes son Decimal. La carpeta `destino` es nueva y ya existe. Este fragmento muestra una agregación y parte de un archivo; faltan campos del resumen y los otros dos productos.

```
ventas = sum((f["ventas"] for f in validas), Decimal("0"))
costos = sum((f["costos"] for f in validas), Decimal("0"))
margen = ventas - costos
resumen = {"ventas": str(ventas), "margen": str(margen)}
with open(destino / "resumen.json", "x", encoding="utf-8") as f:
    json.dump(resumen, f, ensure_ascii=False, indent=2)
```

`sum` reúne los importes de todas las filas válidas. `Decimal("0")` inicia la suma en cero decimal, incluso si la colección está vacía. `str` conserva cada importe como texto para el JSON, conforme al contrato; `json.dump` escribe la estructura en el archivo [3]. El modo `x` exige crear un archivo nuevo y falla si ya existe. La implementación completa debe validar antes el destino de los tres productos.

Actividad individual de lectura

Localiza en tu script una condición, una detención, una suma y un guardado. Anota la regla, la línea o función y una entrada que la pondría a prueba. Explica una de ellas por el chat. Si solo puedes repetir la explicación del agente, vuelve al fragmento y anticipa un resultado pequeño con tus palabras.

2.4 Una biblioteca estándar suficiente

La solución puede utilizar `csv` para leer registros, `decimal` para importes, `json` para resultados, `argparse` para parámetros y `pathlib` para rutas. Son piezas del lenguaje que el agente debe explicar en relación con la tarea. No es obligatorio instalar pandas ni crear servicios externos.

La documentación de `csv` señala que la lectura básica entrega campos como cadenas de texto [1]. Por ello, leer «10» no basta para demostrar que la cantidad sea entera y positiva: convertir y validar son decisiones posteriores.

Para discutir: Si ves una función llamada validar, ¿qué evidencia permite saber que realmente valida todas las reglas?

Idea central

Comprender el código consiste en conectar cada acción con una regla y una salida observable. Un nombre claro ayuda, pero no prueba corrección.

Capítulo 3

El contrato antes del programa

3.1 Entradas, parámetros y salidas

El contrato reúne el objetivo, las fuentes autorizadas, las reglas y los productos esperados. Debe preceder a la generación del código. Si la instrucción solo dice «limpia y resume», el agente puede tomar decisiones incompatibles con el diagnóstico E1.

Las entradas son el CSV original, el diccionario y reglas, el contrato y los archivos de políticas. La fecha simulada de ejecución es un parámetro explícito. La carpeta de salida identifica un destino nuevo; no debe coincidir con una entrada ni ocultar resultados anteriores. La matriz de pruebas permite comprobar aceptación, pero no reemplaza esas entradas.

Elemento	Pregunta que resuelve
CSV y diccionario	¿Qué registros existen y qué significa cada campo?
Contrato	¿Qué se acepta, excluye, detiene y produce?
Políticas y fecha	¿Qué regla de clasificación cubre esta ejecución?
Destino nuevo	¿Dónde se conservará la evidencia sin alterar entradas?
Pruebas	¿Qué resultados permiten aceptar provisionalmente el programa?

3.2 La interfaz no cambia la regla

Un programa puede recibir parámetros desde una línea de comandos. En la interfaz prevista, `--entrada` identifica el CSV, `--politicas` la carpeta de políticas, `--fecha-ejecucion` la fecha y `--salida` el destino. Son nombres para pasar información; no hace falta memorizarlos.

```
python pipeline.py --entrada ventas_original.csv --politicas . --fecha-ejecucion
    2026-10-18 --salida resultados_v1
```

El punto indica la carpeta actual. Si las políticas están en una subcarpeta, el agente debe adaptar la ruta. En algunos entornos se usa `python3`; verificar el intérprete disponible antes de ejecutar. Sin opción de salida, el contrato admite JSON por consola. Esto no convierte la consola en una prueba suficiente de los archivos cuando la entrega exige conservarlos.

3.3 Condiciones de detención

Encabezado distinto, campos inesperados, ID conflictivo o política no determinable detienen la ejecución. El programa debe explicar el problema y devolver una señal de fallo; no continuar con una suposición. Para esos casos no se produce una salida nueva presentada como éxito. Los archivos de una ejecución previa pueden seguir existiendo, pero deben identificarse como anteriores.

Para discutir: ¿Por qué no debe reutilizarse un resumen viejo cuando el comando actual falla?

Capítulo 4

Los dos primeros bloques: leer y validar

4.1 Leer conserva estructura y procedencia

La lectura verifica el archivo correcto y el encabezado exacto. Después asigna a cada registro un origen, empezando en 1 sin contar el encabezado. Ese origen permite regresar desde el reporte al dato recibido. Un ID como V005 representa una clave de venta; el origen 14 representa una aparición concreta. Los dos conceptos no son intercambiables.

En este CSV cada registro ocupa una línea, pero conceptualmente el CSV organiza registros y campos. No conviene tratarlo como texto cortado por comas sin considerar sus reglas de formato. La herramienta elegida por el agente debe respetar el formato y comprobar que cada registro contiene los seis campos esperados.

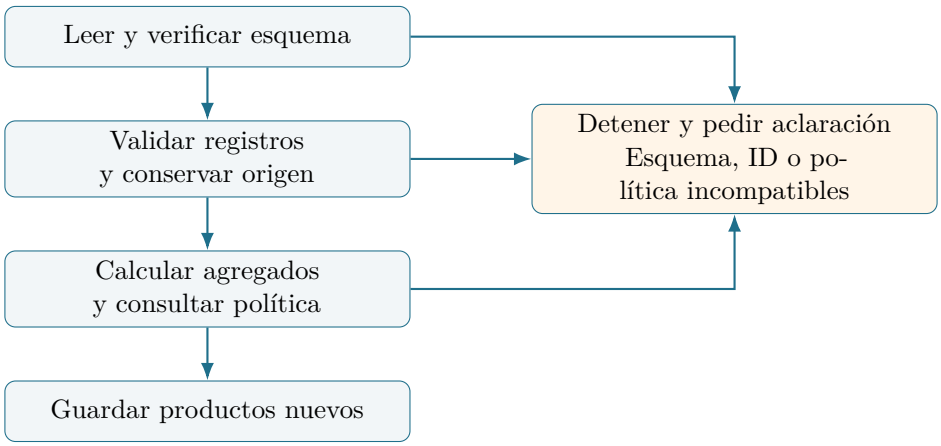
4.2 Validar conserva decisiones y motivos

La validación revisa los valores antes de utilizarlos. Una fecha debe existir en el calendario, no solo tener guiones. Una cantidad debe ser entera, positiva y finita. El precio debe ser positivo; el costo puede ser cero, pero no negativo. Las categorías son valores exactos: normalizarlas silenciosamente modifica el contrato.

La primera V005, origen 5, se acepta. La repetición exacta del origen 14 se excluye y queda ligada a la primera. Si V005 apareciera con otro precio, no se escogería el más reciente ni el mayor: habría conflicto y detención. El significado de «duplicado» depende de la comparación acordada.

Excluir y detener son decisiones diferentes

Una fila con cantidad -2 incumple una regla de valor: se excluye y se registra. Una entrada que cambia `precio_unitario` por `precio` cambia el esquema: el proceso se detiene. La primera incidencia está prevista por el contrato; la segunda requiere revisar qué fuente se recibió.



Lectura del esquema: las filas inválidas previstas se excluyen en validar; las incompatibilidades que exigen aclaración detienen. El camino hacia guardar solo se completa con entradas y política compatibles.

4.3 El tamaño del maestro no limita el pipeline

Los 16 registros identifican el archivo maestro conocido. En la Semana 1 se contrastó ese conteo para comprobar que se estaba diagnosticando la fuente correcta. Esa verificación no autoriza introducir una regla universal que rechace cualquier CSV con un tamaño diferente.

La estructura y las reglas de cada fila se mantienen; el tamaño esperado pertenece a la prueba. El maestro tiene 16 registros; V001 sola tiene uno; P8 contiene dos. Los tres archivos son entradas compatibles si conservan el esquema acordado. El código no debe contener una condición general del tipo «si el número de filas no es 16, detener».

Fuente completa y archivo de prueba

Un archivo de prueba separado no reemplaza el maestro. Registrar su nombre, propósito, conteo esperado y relación con los orígenes originales. En P8, origen local 1 corresponde al original 15 y origen local 2 al original 16. El pipeline numera la entrada que recibe; la bitácora conserva esa correspondencia.

4.4 La conciliación de conteos

En una ejecución que completa la validación, cada registro queda aceptado o excluido. Por tanto:

$$N_{\text{leídas}} = N_{\text{válidas}} + N_{\text{excluidas}}.$$

Esta identidad detecta pérdidas de registros y dobles conteos de incidencias. No basta por sí sola: un programa puede asignar decisiones incorrectas y aun así cumplir la suma. Hay que revisar además los orígenes y motivos. En el caso de referencia, $16 = 13 + 3$; las tres excluidas son 14, 15 y 16.

Para discutir: Una fila tiene precio vacío y costo negativo. ¿Cuántas filas excluidas y cuántos motivos puede generar?

Idea central

Leer verifica la estructura; validar aplica las reglas. Cada decisión debe poder regresar al registro original.

Capítulo 5

Calcular importes, agregados y porcentajes

5.1 La unidad del cálculo

Para una fila válida i, cantidad q_i, precio p_i y costo unitario c_i:

$$V_i = q_i p_i, \qquad C_i = q_i c_i, \qquad M_i = V_i - C_i.$$

Ventas y costos son importes monetarios, no números de filas. Unidades es la suma de cantidades. El origen no participa en las fórmulas: sirve para trazabilidad. El cálculo se realiza solo después de aceptar los registros.

Para una categoría k, sumar únicamente sus filas válidas. Para el total, sumar todas las aceptadas. Ambas rutas deben conciliar: la suma de ventas de las categorías debe ser igual a las ventas totales; lo mismo ocurre con costos, margen y unidades. Esa comprobación puede revelar una categoría olvidada o una fila contada dos veces.

5.2 Dividir después de sumar

$$V = \sum_i V_i, \quad C = \sum_i C_i, \quad M = V - C, \quad R = 100\frac{M}{V}.$$

Si $V > 0$, el porcentaje global mide margen por unidad monetaria vendida. Promediar porcentajes de filas concede a cada fila el mismo peso aunque sus ventas sean distintas. Tampoco se promedian porcentajes de categorías: primero se suman sus importes.

Dos filas que revelan el error

V001: ventas 250, costos 150, margen 100, porcentaje 40 %. V002: ventas 1200, costos 840, margen 360, porcentaje 30 %. Juntas: ventas 1450, costos 990, margen 460; $R = 31,724137\ldots$ %, mostrado 31,72 %. La media simple de 40 y 30 da 35 %, porque ignora que V002 vende más.

La misma razón puede expresarse como media ponderada, con peso V_i/V. Esto explica matemáticamente por qué una venta mayor contribuye más. En el trabajo práctico se pide la razón de sumas porque hace visibles numerador y denominador y evita confundir tipos de promedio.

5.3 Pérdidas y conjunto vacío

Una venta válida con precio menor que costo produce margen negativo. No debe excluirse para mejorar el reporte. Un margen negativo describe el resultado de una venta aceptada; una cantidad negativa describe un dato que incumple el contrato de este caso.

Con cero filas válidas, las sumas monetarias pueden ser cero, pero $0/0$ no define un porcentaje. El resultado es `null` y NO EVALUABLE, no 0 % ni SIN ALERTA. La distinción permite separar ausencia de base de cálculo de una operación con margen efectivamente nulo.

Para discutir: ¿Qué información se pierde si solo se entrega el porcentaje global sin sus importes?

Capítulo 6

Precisión decimal, redondeo y representación

6.1 Representar un importe antes de operar

El contrato utiliza Decimal para las cantidades monetarias. La documentación de Python explica que ciertos decimales no tienen representación exacta en coma flotante binaria y que el contexto decimal controla precisión y redondeo [2]. En el caso, el agente debe construir importes desde el texto original, conservar suficiente precisión y evitar convertirlos primero a `float`.

Decimal no elimina todas las decisiones numéricas. Una división periódica, como 1510/5720, requiere una representación finita en el programa. «Sin redondeos intermedios» significa no recortar importes ni porcentajes a dos decimales antes de agregar o comparar. La precisión interna debe ser suficiente para estas operaciones; el estudiante revisa esa intención y sus pruebas, sin prometer exactitud ilimitada.

6.2 Calcular, comparar y mostrar

El orden es importante: calcular importes y porcentaje con la precisión interna; comparar con el umbral de política; por último, preparar la visualización a dos decimales con la regla de presentación del contrato. El número mostrado ayuda a leer; no reemplaza el valor usado para decidir.

Dos respuestas con el mismo porcentaje mostrado

Con ventas 100 y costos 70, el porcentaje es 30 %: v2 clasifica SIN ALERTA. Con costos 70.001, el porcentaje es 29.999 %: se muestra 30,00 %, pero v2 clasifica ALERTA. La diferencia está en el valor anterior al redondeo, no en el texto de la tabla.

6.3 Conservar Decimal en un JSON

En el contrato, los decimales definidos se serializan como cadenas de texto para conservar su representación; el porcentaje no definido se representa con `null`. No confundir `"null"`, que es texto, con `null`, que expresa ausencia de valor. Tampoco eliminar ese campo y dejar al lector adivinar qué pasó.

```
{"ventas": "1450", "costos": "990", "margen": "460",
 "margen_pct": "31.724137931034482758620689 66",
 "margen_pct_mostrado": "31.72"}
```

Este fragmento corresponde al microejemplo de dos filas y omite otras partes del resumen. No es la salida completa del caso. El contenido debe permitir relacionar precisión interna y presentación, sin alterar el original.

Para discutir: Si 29,999 % se muestra como 30,00 %, ¿qué valor debe utilizar la condición de alerta?

Capítulo 7

La política se consulta, no se inventa

7.1 Dos fechas con responsabilidades distintas

La fecha de una venta describe cuándo ocurrió el hecho ficticio. La fecha simulada de ejecución selecciona la política aplicable a la clasificación del resultado. En este contrato todas las ventas válidas se agregan; sus fechas no se utilizan para elegir reglas de alerta individuales.

v1 cubre del 2026-10-05 al 2026-10-18 inclusive y alerta si el margen global es menor que 25 %. v2 comienza el 2026-10-19, sin cierre declarado, y alerta si es menor que 30 %. La igualdad al umbral siempre es SIN ALERTA. Los operadores y vigencias son parte de la regla, no preferencias del agente.

7.2 Elegir una única política válida

El agente debe leer archivos y comprobar que una sola política cubra la fecha explícita. Si no hay ninguna, hay solapamiento o los campos no permiten interpretar la regla, se detiene. Elegir el archivo con nombre más nuevo no verifica vigencia; extender v1 hasta el día 19 inventa una autorización.

Cambió la clasificación, no el negocio

Microcaso hipotético: ventas 100, costos 73, margen 27, porcentaje 27 %. El 18/10, v1, SIN ALERTA; el 19/10, v2, ALERTA. El cambio se explica por el umbral de 25 a 30. No se puede concluir que las ventas cayeron ni que aumentaron costos: los importes no cambiaron.

7.3 Un estado informativo

ALERTA es una clasificación para revisión humana. No autoriza cambiar precios, borrar registros ni enviar mensajes a gerencia. La respuesta identifica archivo de política, versión, fecha utilizada, porcentaje y operador. Esa cita permite verificar una clasificación sin depender de la memoria del agente.

En el conjunto vacío debe existir una política válida y una fecha explícita para completar el flujo según contrato; el porcentaje sigue siendo indefinido. Una política ausente es una detención distinta de NO EVALUABLE. Mezclar ambas condiciones oculta qué debe recuperarse.

Para discutir: ¿Qué debe conservarse para demostrar que una alerta nueva se debe solo a la política?

Idea central

La política clasifica un cálculo ya definido. Su vigencia se decide por la fecha simulada de ejecución, y el estado no concede permisos comerciales.

Capítulo 8

El cuarto bloque: guardar sin perder procedencia

8.1 Tres productos con funciones diferentes

`ventas_validas.csv` conserva las columnas originales y agrega origen, ventas, costos y margen. Mantiene el orden de las filas aceptadas. Permite revisar el cálculo a nivel de venta. `reporte_calidad.json` conserva conteos e incidencias, con valores originales, motivos y decisión. Permite revisar qué quedó fuera y por qué. `resumen.json` conserva agregados por categoría y total, porcentaje, presentación y política. Permite responder la pregunta analítica.

Ninguno sustituye a los demás. Un resumen correcto no revela si se perdió el origen de una exclusión. Un CSV válido no informa por sí solo qué política se utilizó. Un reporte de calidad completo no prueba que el cálculo monetario sea correcto.

Producto	Comprobación que facilita
Ventas válidas	V005 aparece una vez; las filas conservan origen; cada importe se puede recalcular.
Calidad	Conteos concilian; se explican los orígenes 14, 15 y 16; no se imputaron valores.
Resumen	Categorías concilian con total; porcentaje es razón de sumas; política y fecha son explícitas.

8.2 Una ejecución tiene identidad

Guardar en `resultados_v1` y en otra carpeta tras una corrección evita confundir versiones. Registrar el script, las entradas, los parámetros, el comando real, el intérprete observado y los archivos producidos. Una fecha del sistema puede ayudar a ordenar la bitácora, pero no reemplaza la fecha simulada de política.

La prueba de original intacto puede ser una comparación real antes y después o una huella realmente calculada. Copiar un valor de ejemplo no demuestra integridad. Si la herramienta no tiene acceso al archivo inicial o no ejecutó la comparación, declarar esa limitación.

8.3 R1: repetir la misma versión sin regenerarla

Después de revisar una ejecución base, conservar exactamente el mismo archivo de código. Mantener el CSV, los archivos de políticas, la fecha simulada y el intérprete. Ejecutarlo de nuevo cambiando únicamente el destino. El cambio de carpeta es necesario para conservar ambas evidencias; no cambia los parámetros de negocio.

```
python pipeline.py --entrada ventas_original.csv --politicas . --fecha-ejecucion
    2026-10-18 --salida resultados_base
python pipeline.py --entrada ventas_original.csv --politicas . --fecha-ejecucion
    2026-10-18 --salida resultados_repeticion
```

Son comandos ilustrativos de la interfaz prevista: adaptar el ejecutable y las rutas al entorno disponible, sin alterar las reglas. Registrar los comandos realmente usados. Comparar el archivo del script o su

huella antes y después permite documentar que no se regeneró ni modificó entre ambas ejecuciones.

Debe coincidir	Puede diferir si se documenta
Filas válidas, orden y orígenes.	Carpeta de salida.
Conteos, exclusiones, motivos e importes.	Hora o identificador real de ejecución.
Agregados, porcentaje, política y estado.	Rutas incluidas como metadatos.

Comparar el contenido interpretado de los CSV y JSON, no solamente el tamaño del archivo ni una captura. Diferencias de espacios en JSON no cambian su contenido. Los campos variables se identifican de forma explícita; no se permite ignorar un importe o una decisión diferente para declarar éxito.

La repetición comprueba estabilidad bajo las condiciones utilizadas; un error también puede repetirse. Por eso R1 complementa P1–P8 y no las reemplaza. Tampoco demuestra todavía portabilidad a otra computadora o entorno: ese será el puente hacia versiones y Docker.

8.4 Abrir los archivos también es revisar

La revisión incluye descargar o abrir productos, no solo leer la respuesta del chat. Comprobar encabezados, orígenes, `null`, versiones y números. Si el agente afirma que guardó tres archivos pero solo existe uno, la tarea está incompleta aunque el relato describa los tres.

Para discutir: ¿Cuál de los tres productos abrirías para investigar por qué se excluyó el origen 16?

Capítulo 9

Probar antes de aceptar y después de corregir

9.1 Entrada, esperado, observado y evidencia

Una prueba concreta identifica una entrada, una regla, el resultado esperado y la salida observada. El esperado proviene del contrato y del cálculo independiente; no debe copiarse de la misma salida que se está examinando. De lo contrario, el programa se convierte en juez de sí mismo.

La matriz `matriz_8_pruebas.csv` deja vacíos los campos de observado y evidencia para que cada estudiante registre su trabajo. «Pasaron todas» es una afirmación; un archivo de salida y una comparación permiten comprobarla. Si una prueba fue docente observada, conservar la salida compartida y declarar quién la ejecutó.

Prueba	Qué revisa	Contraste clave
P1	Conjunto original	Leídas = válidas + excluidas; decisiones explicadas.
P2	Repetición V005	Primera conservada, segunda excluida, importe una vez.
P3	Origen 15	Cantidad negativa excluida sin valor absoluto.
P4	Origen 16	Precio vacío excluido sin imputación.
P5	Agregados	Unidades, ventas, costos y margen por categoría y total.
P6	Porcentaje	Razón de sumas y contraste independiente.
P7	Fechas y frontera	Política por ejecución; igualdad y comparación antes de redondear.
P8	Solo orígenes 15/16	Cero válidas, porcentaje `null`, NO EVALUABLE.

9.2 Agrupar pruebas para comprenderlas en 90 minutos

P1–P6 pueden contrastarse sobre los mismos productos de la ejecución maestra: primero conteos y exclusiones, después agregados y porcentaje. No hace falta generar seis programas ni tratar cada fila de la matriz como una conversación nueva. El microejemplo V001/V002 aporta una comprobación aritmética independiente para P6.

P7 se organiza en subcomprobaciones: maestro al 18/10, maestro al 19/10, igualdad 30 % y valor 29,999 % mostrado como 30,00 %. P8 usa el subconjunto de dos registros inválidos. Cada subcomprobación conserva su entrada, fecha, salida y contraste. Una evidencia puede apoyar varias pruebas, pero no se aprueba una prueba por analogía si no se observó su resultado.

9.3 D1: una detención visible, paso a paso

Antes de la clase, el docente prepara un CSV separado con la fila V001 y cambia únicamente el nombre de columna `precio_unitario` por `precio`. Conserva la entrada maestra. La clase predice el resultado antes de observarlo.

1. Ejecutar el mismo script con ese archivo de prueba, políticas válidas y fecha 2026-10-18, hacia una carpeta nueva.

2. Comprobar mensaje de esquema incompatible y señal de fallo del comando; registrar ambos realmente observados.
3. Verificar que la ejecución no se presenta como exitosa ni entrega un nuevo resumen de éxito. Una carpeta creada o un registro de error no equivalen a éxito.
4. Identificar cualquier producto anterior como tal; no usarlo para responder sobre esta ejecución.

El docente realiza la demostración; cada estudiante explica por el chat por qué corresponde detener y no renombrar automáticamente. La evidencia se incorpora a la bitácora como D1. No se modifica la matriz P1–P8 ni la rúbrica. Si se usa como alternativa un ID repetido con distinto contenido, el contrato también exige detener; documentar cuál de las dos situaciones se mostró.

9.4 Lo que la matriz no cubre por sí sola

El contrato también exige detener ante esquema nuevo, ID conflictivo y política no determinable. Revisar estos controles con copias y pruebas pequeñas complementa la matriz; ocho pruebas aprobadas no demuestran todos los casos posibles.

La frontera separa $<$ de $\leq$; el conjunto vacío detecta división por cero o clasificación engañosa; una venta a pérdida detecta exclusión indebida. Cada prueba debe revelar una clase de error mediante una comparación concreta.

9.5 Corrección mínima y regresión

Si el agente promedió porcentajes, corregir esa implementación, conservar una nueva versión y ejecutar el microejemplo que reveló el fallo. Después volver a ejecutar las ocho pruebas base: una modificación puede resolver un error y crear otro. Eso es regresión. No cambiar el esperado para que coincida con el resultado ni modificar el original para obtener una aprobación.

Si no hubo fallo, documentar aceptación provisional sustentada en evidencia; no inventar una corrección para completar la bitácora. Pruebas aprobadas permiten aceptar dentro de lo comprobado, no afirmar infalibilidad general.

Para discutir: ¿Por qué el esperado y el observado deberían tener orígenes distintos?

Capítulo 10

Generar, revisar y corregir con agentes

10.1 Una instrucción que hace revisable el trabajo

El prompt inicial debe incluir rol, entradas, contrato, cuatro bloques, productos, límites y pruebas. El agente presenta un plan antes de ejecutar en el entorno autorizado. El estudiante revisa qué archivos se leerán y escribirán; la autorización del caso no abre otras carpetas ni servicios.

```
Actúa como agente programador de Comercial Horizonte. Lee solo CSV, diccionario, contrato,
    políticas y matriz adjuntos. Genera pipeline.py breve con biblioteca estándar y
    Decimal, en bloques leer-validar-calcular-guardar. Explica cada bloque y localiza sus
    decisiones. El tamaño 16 identifica solo el maestro: admite otros CSV compatibles y
    declara el conteo esperado de cada prueba. Conserva original y origen; aplica
    exclusiones exactas, sin imputación ni abs. Detén ante esquema nuevo o ID conflictivo.
    Exige fecha simulada y política única válida. Suma antes de dividir y compara antes
    de mostrar dos decimales. Produce los tres archivos y JSON consola. Presenta plan,
    destinos nuevos y pruebas. Si no puedes ejecutar, declara el límite y prepara
    evidencia pendiente para docente.
```

10.2 Revisar con preguntas que buscan fallos

El revisor debe comparar código y contrato, no valorar solo legibilidad. Preguntar dónde se detecta conflicto de ID, cómo se verifican números finitos, qué sucede sin política y qué evidencia demuestra original intacto. Puede ser otra conversación o un segundo rol, pero eso no garantiza independencia de errores.

```
Actúa como revisor. Contrasta el script con contrato y P1-P8. Para cada prueba muestra
    entrada, esperado independiente, observado y evidencia real. Busca promedio de
    porcentajes, redondeo temprano, imputación, pérdida excluida, duplicado contado y polí
    tica elegida por fecha de venta. Distingue lectura del código de ejecución. No
    apruebes una prueba pendiente. Señala la corrección mínima sin alterar reglas ni
    entradas.
```

10.3 Corregir a partir de una discrepancia

Una petición de corrección debe describir el fallo observado y el criterio violado. Por ejemplo: «el microejemplo devuelve 35 %; el contrato exige 100(460/1450)». La evidencia permite una reparación concreta. Pedir «mejora el código» puede introducir cambios que dificultan localizar la causa.

```
Corrige solo el fallo documentado: [entrada, esperado y observado]. Explica la causa y
    guarda nueva versión. No cambies contrato, original ni esperado. Repite la prueba
    fallida y P1-P8 con herramienta si está disponible. Entrega comando y nuevos productos
    ; distingue aprobado, fallido, observado y pendiente. Si no hubo ejecución, no afirmes
    éxito.
```

Para discutir: ¿Qué tres comprobaciones conservarías aunque el segundo rol diga que el código está correcto?

Capítulo 11

La práctica de la Clase 4 y la entrega E2

11.1 Una sesión de 90 minutos con productos concretos

Minutos	Trabajo y resultado
0–8	Preparar entradas y anticipar resultados.
8–23	Generar y leer las decisiones esenciales.
23–38	Ejecutar base y revisar P1–P6 sobre sus productos.
38–53	P7: cambios de política e igualdad/frontera.
53–63	P8 y demostración docente de detención D1.
63–73	Repetición R1 del mismo script conservado.
73–83	Corrección focalizada y prueba del fallo, si corresponde.
83–90	Cierre E2, explicación individual y pendientes.

La ruta principal utiliza un entorno con acceso a los archivos y ejecución de Python. Si el estudiante solo puede generar código, el docente puede ejecutarlo y compartir productos. Debe registrarse esa diferencia. No se requiere API, secretos ni una instalación de dependencias para este caso.

11.2 Preparación docente y contingencia

El docente prepara los archivos pequeños del apéndice y comprueba que dispone de una vía de ejecución. La clase no se consume transcribiendo CSV ni instalando dependencias. Cada estudiante conserva su solución propia; la referencia docente permite mostrar productos si la herramienta individual no ejecuta, con esa modalidad expresamente identificada.

Los bloques de la agenda son un presupuesto de trabajo, no una promesa de que toda generación o corrección termine a tiempo. Si surge un bloqueo, registrar la causa y continuar la demostración con evidencia docente. No declarar aprobadas pruebas que quedaron pendientes. Conservar también la versión que falló para explicar el problema.

11.3 Dos horas autónomas, con alcance definido

Tiempo orientativo	Trabajo
0–30 min	Completar lectura del código y correspondencia entre reglas y decisiones.
30–75 min	Resolver fallos y completar P1–P8; si cambió el código, ejecutar regresión completa.
75–100 min	Comparar productos y completar evidencia de R1 y D1 ejecutada u observada.
100–120 min	Ordenar E2 y redactar explicación individual de resultados y límites.

No se agregan ocho pruebas nuevas ni otra entrega. D1 y R1 se registran en la bitácora existente. Si una limitación impide completar una comprobación, debe permanecer visible como pendiente y coordinarse su resolución o evidencia docente.

11.4 Qué conserva E2

E2 incluye E1, el script generado y revisado, las ventas válidas, calidad, resumen, matriz con ocho pruebas y bitácora. Conservar prompts, comandos y la modalidad de ejecución. Registrar D1 y R1 en la bitácora, con el script conservado y las dos carpetas comparadas. El estudiante verifica al menos el microejemplo, los orígenes de exclusión y el cálculo global; el docente ayuda a contrastar los agregados completos.

La explicación final responde qué definió el estudiante, qué generó el agente, qué se ejecutó realmente, qué error se buscó y qué se corrigió o aceptó provisionalmente. No basta decir «usé IA»: hay que describir el trabajo y la revisión.

11.5 Cómo se conecta con la Semana 3

El analista reutilizable consumirá estos productos. Si el resumen carece de procedencia o contiene un porcentaje incorrecto, una respuesta fluida amplificará el problema. Por ello, entregar E2 completo prepara una base verificable para la conversación posterior. El agente analista no sustituirá el pipeline por números recordados de este libro.

Revisión de cierre

¿El original sigue intacto? ¿Los tres productos existen? ¿V005 participa una vez? ¿Los conteos y categorías concilian? ¿El porcentaje es razón de sumas? ¿La política cubre la fecha? ¿Las pruebas tienen evidencia? ¿Se distingue ejecución individual, observada y pendiente? Cada respuesta debe poder señalar un archivo o una comprobación.

Idea central

E2 entrega un proceso y evidencia para revisarlo. La calidad del código se juzga por fidelidad al contrato, cálculo comprobable y límites conservados.

Apéndice A

Referencia del caso y contraste independiente

A.1 Reglas que permanecen durante las cinco semanas

El encabezado debe ser exactamente `id,fecha,categoria,cantidad,precio_unitario,costo_unitario`, en ese orden. Un cambio de esquema, un número inesperado de campos o un ID repetido con contenido distinto exige detención y aclaración. No se renombra una columna ni se resuelve un conflicto por intuición.

El ID debe existir; la fecha debe ser una fecha real de calendario en formato AAAA-MM-DD; la categoría debe coincidir con uno de los tres valores admitidos. La cantidad debe ser entera, positiva y finita; el precio, positivo y finito; el costo, no negativo y finito. Precio menor que costo es una pérdida válida, no un defecto de calidad. No imputar un precio, usar valor absoluto ni redondear una cantidad para hacerla válida.

Ante un duplicado exacto de los seis campos originales, se conserva la primera aparición y se excluyen las posteriores. Una fila inválida puede tener varios motivos: registrar todos los identificados. El número de motivos no se suma como si fuera número de filas excluidas. El original nunca se modifica; origen significa posición del registro de datos, empezando en 1 y sin contar el encabezado.

A.2 CSV original del caso

```
id,fecha,categoria,cantidad,precio_unitario,costo_unitario
V001,2026-10-05,Oficina,10,25,15
V002,2026-10-05,Tecnologia,2,600,420
V003,2026-10-06,Hogar,5,80,56
V004,2026-10-06,Oficina,20,12,8
V005,2026-10-07,Tecnologia,3,150,120
V006,2026-10-07,Hogar,4,100,85
V007,2026-10-08,Oficina,8,30,18
V008,2026-10-08,Tecnologia,1,900,720
V009,2026-10-09,Hogar,6,50,35
V010,2026-10-09,Oficina,15,20,14
V011,2026-10-12,Tecnologia,2,250,200
V012,2026-10-12,Hogar,3,120,96
V013,2026-10-13,Oficina,12,15,9
V005,2026-10-07,Tecnologia,3,150,120
V014,2026-10-13,Hogar,-2,75,50
V015,2026-10-14,Oficina,7,,11
```

Las filas 1 a 13 se aceptan. El origen 14 repite exactamente el origen 5: mantener V005 una sola vez. El origen 15 tiene cantidad -2 y se excluye; no consta una regla de devoluciones. El origen 16 tiene precio vacío y se excluye; no se completa con cero, promedio ni consulta externa. Por tanto, $16 = 13 + 3$.

A.3 Resultados de referencia: consultar después de calcular

Estos importes se derivan de los 13 registros aceptados. Son referencias para el contraste, no evidencia de que un estudiante ejecutó su programa. Los importes están en UM; las unidades son cantidades de productos.

Categoría	Registros	Unidades	Ventas	Costos	Margen
Oficina	5	65	1210	772	438
Tecnologia	4	8	3050	2320	730
Hogar	4	18	1460	1118	342
Total	13	91	5720	4210	1510

Los porcentajes mostrados son 36,20 %, 23,93 % y 23,42 %, respectivamente. El margen global es $100(1510/5720) = 26,398601\ldots$ %, mostrado como 26,40 %. Se obtiene dividiendo las sumas, no promediando porcentajes de categorías ni de filas. El porcentaje mostrado es una presentación: la comparación utiliza el valor anterior al redondeo.

A.4 Políticas y estados de referencia

Versión	Vigencia simulada inclusiva	Regla del margen global
v1	2026-10-05 a 2026-10-18	ALERTA si porcentaje sin redondear < 25; igualdad: SIN ALERTA.
v2	Desde 2026-10-19, sin cierre	ALERTA si porcentaje sin redondear < 30; igualdad: SIN ALERTA.

Para el conjunto completo: ejecución 2026-10-18, v1, SIN ALERTA; ejecución 2026-10-19, v2, ALERTA. Ventas, costos, margen y exclusiones permanecen iguales. La regla se aplica al margen global: no inventar alertas por categoría bajo estas políticas.

Sin fecha explícita, antes de la primera vigencia, sin archivo que cubra la fecha, con políticas solapadas o mal formadas: detener y solicitar aclaración. No ampliar una vigencia ni elegir por fecha de venta. Con cero filas válidas o ventas cero, el porcentaje es `null` y el estado NO EVALUABLE, aunque las sumas monetarias del conjunto vacío sean cero.

A.5 Fixtures pequeños que aíslan una regla

Un fixture es una entrada de prueba separada del original. Para igualdad con v2, usar cantidad 1, precio 100, costo 70: margen 30 y porcentaje 30 %, SIN ALERTA. Para redondeo temprano, cambiar solo costo a 70.001: margen 29.999, porcentaje 29.999 %, mostrado 30,00 %, pero ALERTA. En el CSV de prueba el separador decimal es punto; en la explicación española se usa coma.

Para el caso sin filas válidas, copiar únicamente los registros originales 15 y 16, conservando encabezado. Su numeración local es 1 y 2; anotar correspondencia local 1 $\leftrightarrow$ original 15 y local 2 $\leftrightarrow$ original 16. Resultado: dos leídas, cero válidas, dos excluidas; porcentaje `null`, NO EVALUABLE. Ningún fixture reemplaza el CSV maestro.

A.6 Archivos de prueba listos para preparar

Estos ejemplos son independientes y llevan encabezado propio. Se guardan como archivos nuevos en una carpeta de pruebas. Las fechas son ficticias; para frontera y redondeo usar ejecución 2026-10-19

y política v2. No juntar las filas de dos pruebas en un mismo CSV: se alteraría el agregado que permite aislar la regla.

Frontera: `frontera_30.csv`. Una fila válida, ventas 100, costos 70, margen 30, porcentaje 30 %, SIN ALERTA.

```
id,fecha,categoria,cantidad,precio_unitario,costo_unitario
T030,2026-10-14,Oficina,1,100,70
```

Redondeo: `redondeo_29999.csv`. Una fila válida, ventas 100, costos 70.001, margen 29.999, mostrado 30,00 %, ALERTA.

```
id,fecha,categoria,cantidad,precio_unitario,costo_unitario
T029,2026-10-14,Oficina,1,100,70.001
```

P8: `sin_validas.csv`. Dos leídas, cero válidas, dos excluidas. Orígenes locales 1/2 corresponden a originales 15/16. Con fecha 2026-10-18 y v1 válida, porcentaje `null`, NO EVALUABLE.

```
id,fecha,categoria,cantidad,precio_unitario,costo_unitario
V014,2026-10-13,Hogar,-2,75,50
V015,2026-10-14,Oficina,7,,11
```

D1: `esquema_incompatible.csv`. La única modificación respecto de V001 es el nombre de una columna. Se espera detención, no renombrado automático ni resumen exitoso.

```
id,fecha,categoria,cantidad,precio,costo_unitario
V001,2026-10-05,Oficina,10,25,15
```

La cantidad de registros no es la causa de D1: un archivo de una fila con encabezado correcto es admisible. El error es el esquema incompatible. Todos estos resultados son *esperados de referencia*; solo la ejecución real permite completar el campo observado.

Apéndice B

Actividades y respuestas razonadas

B.1 Actividades para resolver antes de consultar

1. Explicar, con una frase por bloque, qué debe comprobar leer, validar, calcular y guardar. Señalar una salida observable de cada uno.
2. Recalcular V001 y V002. Comparar razón de sumas y media de porcentajes, y explicar por qué difieren.
3. Una fila tiene cantidad 0 y precio vacío. ¿Cuántas filas se excluyen? ¿Qué motivos deben registrarse?
4. Crear en una copia un fixture con cantidad 1, precio 100 y costo 70.001. Predecir porcentaje mostrado y estado con v2.
5. Explicar la diferencia entre una fila inválida, un ID conflictivo y un encabezado cambiado. Proponer la decisión correcta para cada uno.
6. Ejecutar u observar P8 con correspondencia local/original. ¿Por qué porcentaje `null` no equivale a 0 %?
7. Un script produce ventas 6170 en el caso completo. Investigar primero si V005 se contó dos veces. ¿Qué importe aporta esa fila?
8. Una prueba falla y el agente cambia el umbral de 30 a 26. ¿Ha corregido implementación o alterado una regla? Justificar.
9. El agente afirma «pasó P7» sin comando ni salida. Redactar qué evidencia falta.
10. Elegir tres comprobaciones humanas que permitan defender E2 sin memorizar sintaxis.
11. Explicar por qué un pipeline que exige siempre 16 filas impide realizar P8 correctamente.
12. Comparar dos ejecuciones del mismo script: cambió la hora, pero coinciden filas e importes. ¿Puede pasar R1? ¿Y si cambió el margen?
13. D1 detiene el comando, pero existe un resumen de ayer. Explicar qué evidencia es válida para la ejecución actual.

B.2 Respuestas de referencia

1. Leer verifica archivo, esquema y orígenes; validar asigna aceptar/excluir o detener según reglas; calcular deriva importes, agregados y política; guardar conserva tres productos nuevos con procedencia. Cada afirmación requiere salida o control visible, no solo nombre de función.

2. Ventas 1450, costos 990, margen 460, global 31,724137… %. La media simple es 35 %; concede igual peso a 250 y 1200 UM. La razón de sumas mide margen sobre ventas totales.

3. Una fila excluida con dos motivos. La conciliación cuenta decisiones por registro, no incidencias individuales.

4. Margen 29.999, porcentaje 29.999 %, mostrado 30,00 %. ALERTA porque el valor usado por $<$ es menor que 30.

5. Valores inválidos se excluyen y explican. ID repetido con contenido distinto y encabezado cambiado detienen: requieren una resolución humana del contrato o de la fuente.

6. Dos leídas, cero válidas, dos excluidas. `null` expresa porcentaje indefinido; 0 % afirmaría un cálculo definido con margen cero y ventas positivas.

7. V005 aporta ventas $3 \times 150 = 450$. $5720 + 450 = 6170$, por lo que la hipótesis es compatible con ese error. No basta la coincidencia: comprobar CSV válido, orígenes y código antes de concluir causa.

8. Alteró una regla. Una corrección válida conserva umbral y repara el cálculo o la comparación que incumplía el contrato.

9. Faltan fecha y política utilizadas, fixture de frontera, comando real, porcentaje calculado/mostrado, estado y contraste humano. Declarar pendiente hasta obtener evidencia.

10. Ejemplos: cálculo V001/V002, exclusiones 14–16 y razón global. La elección debe explicar qué error detecta y dónde se conserva la comprobación.

11. El tamaño esperado identifica una entrada concreta. P8 tiene dos registros y conserva el esquema: debe llegar a validar, excluir ambas filas y producir NO EVALUABLE. Rechazarlo por no tener 16 confunde el maestro con el contrato general.

12. Puede pasar si solo difieren metadatos identificados, se conservó el script y todos los resultados de negocio coinciden. Un margen diferente exige investigar código, datos, reglas o parámetros y no aprobar R1.

13. El mensaje y la señal de fallo actuales documentan la detención. El resumen de ayer pertenece a otra ejecución. Para aceptar D1 también se comprueba que no se presentó un nuevo resultado exitoso.

B.3 Taller de revisión: un fallo que parece razonable

Supongamos que el agente entrega un script que conserva V005 una vez, excluye 15/16 y calcula importes correctamente, pero toma la media simple de los tres porcentajes de categoría. El total mostrado se aproxima a 27,85 %: parece compatible con el caso y hasta produce los mismos estados v1/v2 que la referencia. No obstante, la fórmula incumple el contrato. Acertar una clasificación no demuestra que el porcentaje esté correcto.

La revisión empieza recuperando los agregados. Oficina aporta ventas 1210 y margen 438; Tecnologia, 3050 y 730; Hogar, 1460 y 342. La fórmula correcta utiliza margen 1510 y ventas 5720. La media de porcentajes concede el mismo peso a cada categoría, pese a sus ventas diferentes. La discrepancia debe anotarse en P6, aunque P7 parezca pasar.

El pedido de corrección describe fórmula observada y esperada, conserva código inicial y solicita sumar antes de dividir. El agente guarda una versión nueva, repite P6 y el microejemplo V001/V002, y luego las ocho pruebas. La aceptación se basa en salidas nuevas; cambiar el texto explicativo sin ejecutar de nuevo no resuelve la evidencia.

Este taller muestra una limitación de probar solo estados finales. Dos cálculos distintos pueden caer del mismo lado del umbral. Por eso la matriz incluye importes, razón de sumas y frontera, además de ALERTA/SIN ALERTA.

B.4 Casos adicionales de lectura crítica

Entrada o conducta hipotética	Decisión y comprobación
Cantidad 1, precio 100, costo 120	Venta válida a pérdida: margen −20; no excluir por signo del margen.
Cantidad 2.5	Excluir por no ser entera; no redondear a 2 o 3.
Precio NaN o Infinity	Excluir como valor no finito; convertir a Decimal no basta.
Mismo ID con precio distinto	Detener por conflicto, aunque cada fila aislada parezca válida.
Políticas v1/v2 solapadas	Detener y aclarar; no escoger según nombre.

Estos casos se realizan en fixtures separados si se revisan con herramienta. Son extensiones pequeñas para comprender el contrato; no reemplazan P1–P8 ni modifican la entrega obligatoria.

Apéndice C

Plantillas y glosario de trabajo

C.1 Registro de prueba

```
Prueba y versión del script:
Entrada o fixture; origen local/original:
Regla y esperado independiente:
Comando real o ejecución docente observada:
Salida producida y ubicación:
Resultado observado:
Coincidencia/discrepancia y explicación:
Estado: aprobado / fallido / observado / pendiente
Revisión humana; corrección y nueva ejecución:
```

C.2 Bitácora de una ejecución

```
Objetivo; archivos autorizados; versión del contrato:
Script y cambios respecto a la versión anterior:
Fecha simulada; política elegida y vigencia:
Herramienta y Python realmente observados:
Comando; carpeta nueva de salida:
Conteos; exclusiones; agregados:
Prueba real de original intacto:
P1-P8: estado, evidencia, modalidad y pendientes:
D1: entrada, fallo real y ausencia de nuevo resultado exitoso:
R1: misma version, entradas, parametros y entorno; dos destinos:
R1: campos comparados, diferencias y metadatos variables:
Qué hizo el agente / qué comprobó el estudiante:
```

C.3 Vocabulario esencial

Script. Archivo que el intérprete puede ejecutar.

Pipeline. Secuencia ordenada desde entrada hasta productos.

Contrato. Reglas, entradas, límites y salidas acordados.

Fixture. Entrada separada que permite aislar una prueba.

Origen. Posición de un registro en la fuente recibida.

Agregado. Suma de una magnitud sobre registros definidos.

Decimal. Representación decimal usada para importes del caso.

Regresión. Repetición de pruebas anteriores después de un cambio.

Repetición R1. Nueva ejecución del mismo script con las mismas condiciones de negocio y comparación de resultados.

Detención D1. Demostración de una entrada incompatible que exige fallo explícito.

Trazabilidad. Relación verificable entre resultado, fuente, reglas y ejecución.

Evidencia. Producto o registro que permite contrastar una afirmación.

NO EVALUABLE. Estado de porcentaje indefinido por ausencia de base válida; distinto de detención por contrato incompatible.

C.4 Criterios comunes de evaluación

Cada entrega semanal se evalúa sobre 20 puntos, según la plantilla del caso. El producto es acumulativo: conservar las evidencias previas y agregar las nuevas.

Dimensión	Puntos	Qué permite comprobar
Definición	5	Objetivo, entradas, contrato, límites y permisos comprendidos.
Producto	5	Archivos descargables, completos y con versiones identificables.
Verificación	5	Pruebas y contraste manual, original intacto, errores visibles.
Explicación del uso de IA	5	Herramienta y prompts reales, quién ejecutó, revisión humana y límites.

Una respuesta bien redactada no reemplaza una prueba ausente. Si una ejecución fue observada, se evalúa la comprensión y evidencia de esa modalidad, sin atribuir al estudiante una ejecución individual inexistente. Un archivo de referencia tampoco demuestra que se generó una solución propia.

Referencias y recursos

[1] Python Software Foundation. *csv: CSV File Reading and Writing*. Documentación oficial. https://docs.python.org/3/library/csv.html.

[2] Python Software Foundation. *decimal: Decimal fixed-point and floating-point arithmetic*. Documentación oficial. https://docs.python.org/3/library/decimal.html.

[3] Python Software Foundation. *json: JSON encoder and decoder*. Documentación oficial. https://docs.python.org/3/library/json.html.

Las reglas operativas provienen de los recursos del proyecto: `diccionario_y_reglas.md`, `contrato_pipeline.md`, `matriz_8_pruebas.csv`, `politica_margen_v1.md`, `politica_margen_v2.md`, `plantilla_bitacora.md` y `plantilla_entrega.md`, en `06_Recursos_del_Caso`. Para contingencia y corrección docente existe `07_Guia_Docente/pipeline_referencia.py`; el estudiante primero genera y revisa su solución.